Remoção de fundo de imagens por corte mínimo em grafos

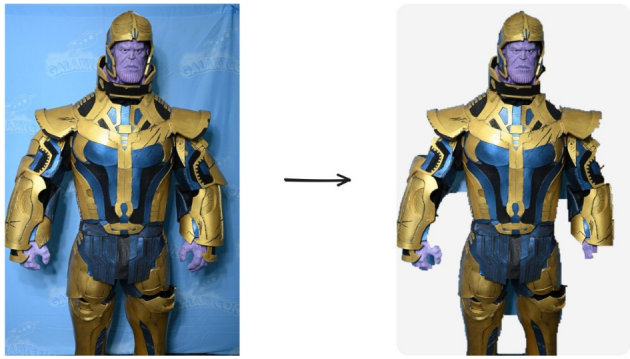
Amanda Bialer · 24.01621-7
Ruy Monteiro · 24.00846-0

ECM306 · Tópicos Avançados em Estrutura de Dados

Título e integrantes. O nome Thanos só aparece no próximo slide.



Aqui entra a sua própria foto, antes e depois. Pergunta para a turma: como o computador sabe o que é fundo?



Aqui entra a sua própria foto, antes e depois. Pergunta para a turma: como o computador sabe o que é fundo?

Isso já existe.

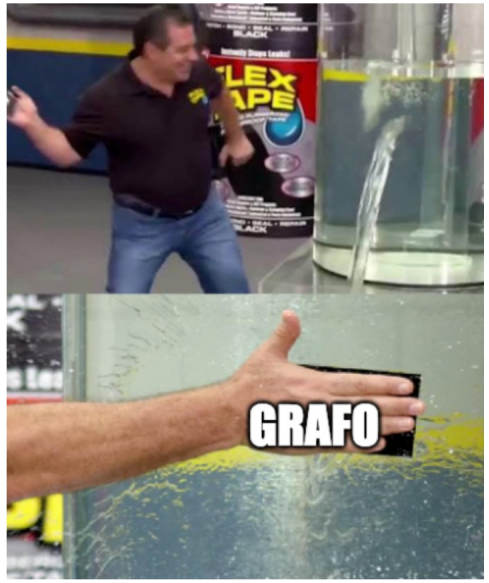
remove.bg **iPhone** **Photoshop** **Canva**

Todas usam redes neurais treinadas com milhões de fotos.

Reconhecer que o problema é resolvido no mercado. A pergunta é: dá para resolver com o que aprendemos na disciplina?

A gente fez com um grafo

Sem rede neural.
Sem biblioteca pronta.
Sem um projeto genérico de mapa.



Uma implementação mais simples e didática. E não é mais um projeto de menor caminho em mapa.

Como funciona?

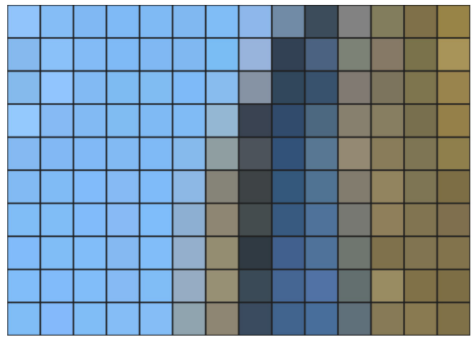
./thanos ui → abrir uma foto de samples (chupeta, wazowski, professor) → riscar objeto e fundo → Snap → mexer nos sliders. Depois o terminal: ./thanos cut samples/wazowski/photo.png --scribbles samples/wazowski/scribbles.png --out wazowski.png. Só então voltar para a teoria.

Uma foto é uma grade de pixels



Vamos dar um zoom. O quadradinho vermelho é a borda do capacete. Ampliado, vira 140 quadrados coloridos, 14 por 10: azul do fundo à esquerda, dourado do capacete à direita.

Cada pixel vira um vértice



pixel = vértice

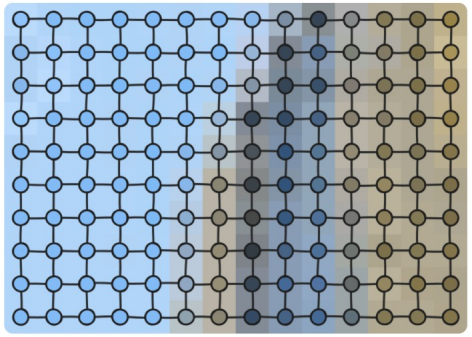
também chamado de nó · *vertex*, *node*

vizinhos = aresta

também chamada de arco · *edge*

Nomenclatura do livro: vértice ou nó (vertex, node); aresta ou arco (edge). Cada pixel se liga aos 4 vizinhos: cima, baixo, esquerda, direita. A foto inteira vira um grafo em grade. Tamanhos e números ficam para depois.

Cada aresta tem um peso



Vizinhos parecidos andam de mãos dadas.

Cores **parecidas**: peso alto, aresta grossa.

Cores **diferentes**: peso baixo, aresta fina.

---

peso · *weight*

Analogia: o peso é a força com que dois vizinhos se seguram. Parecidos estão de mãos dadas com força; na fronteira azul/dourado as mãos mal se tocam. Repare que a borda do objeto já aparece no grafo, só pelos pesos. Em redes de fluxo o peso também se chama capacidade (capacity); esse nome volta mais adiante.



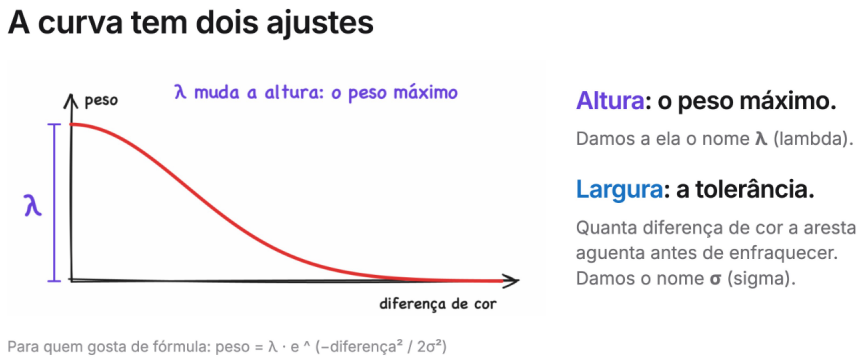
Cores iguais: peso máximo. Cores muito diferentes: peso quase zero.

Só a ideia, sem fórmula. À esquerda, dois pixels vizinhos. Quando a cor do segundo se afasta da do primeiro, a linha entre eles afina e o ponto desce a curva. SE PERGUNTAREM por que é uma curva e não uma reta: (1) diferenças pequenas são ruído, textura e iluminação, não borda; por isso o topo é quase plano e o peso mal cai no começo. (2) Depois que as cores já são claramente diferentes, tanto faz se a diferença é 150 ou 250: é borda do mesmo jeito; por isso a curva encosta no zero e fica lá. Uma reta cruzaria o zero num ponto arbitrário e daria peso negativo depois. (3) O formato de sino é a curva gaussiana, a mesma da distribuição normal: ela modela o ruído da câmera e é a escolha do artigo original de Boykov e Jolly (2001).



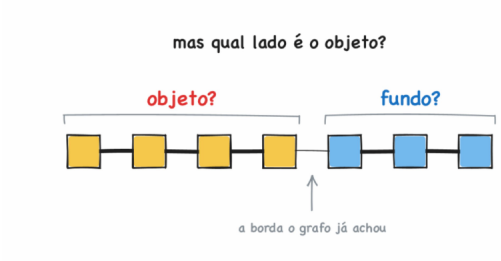
Cada pixel são três números de 0 a 255: vermelho, verde e azul. É só subtrair, canal por canal.

A cor está em RGB. Hexadecimal é só outro jeito de escrever os mesmos três números: F5 é 245, C8 é 200, 4C é 76. Para juntar as três subtrações num número só: eleva cada uma ao quadrado, soma e tira a raiz (a distância entre as duas cores). Exemplo de cima: 25 + 100 + 16 = 141, raiz ≈ 12. Exemplo de baixo: 16 641 + 225 + 26 244 = 43 110, raiz ≈ 208.



A mesma curva do slide anterior, agora com os dois botões que a moldam. Primeiro a altura: é o peso de dois pixels idênticos, o máximo possível; chamamos de lambda. Depois a largura: curva estreita quer dizer que qualquer diferença pequena já derruba o peso; curva larga quer dizer que a aresta tolera bastante diferença; chamamos de sigma. São só nomes para altura e largura. Esses dois valores viram os dois controles da janela, que aparecem mais adiante. A linha tracejada cinza é a curva padrão, para comparar.

Até aqui, o grafo acha a borda. Mas não sabe quem é quem.



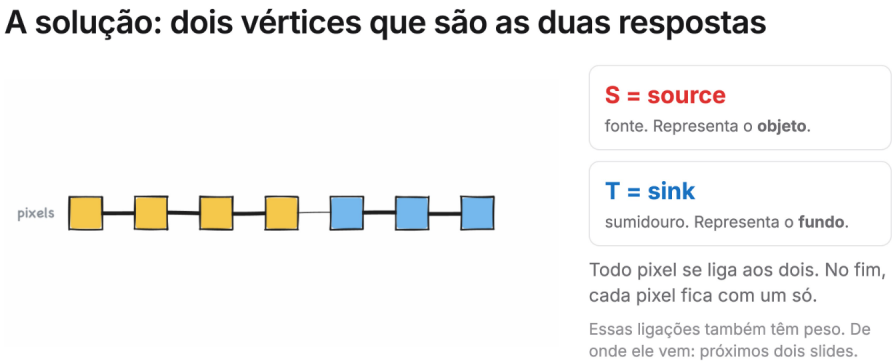
O que já temos

Pixels ligados aos vizinhos. Peso alto entre parecidos, peso baixo na borda.

O que falta

Dizer qual lado é o objeto e qual é o fundo. Só pelos pesos, as duas respostas valem igual.

Pausa para recapitular. Com vértices, arestas e pesos, o grafo já mostra onde estão as bordas: são as arestas finas. Mas borda não é recorte. Uma foto tem muitas bordas (capacete, sombra, dobra da armadura), e nada no grafo diz que o amarelo é o objeto e o azul é o fundo; podia ser o contrário. Falta colocar dentro do grafo as duas respostas possíveis, objeto e fundo. É para isso que servem os dois vértices do próximo slide.



S e T não são pixels: são dois vértices novos, um para cada resposta. S vem de source (fonte) e representa o objeto. T representa o fundo; o nome em inglês é sink (sumidouro), e a letra T vem de target ou terminal, porque S já estava ocupado. É a notação padrão do livro: rede s-t. Pense em dois times: S é o capitão do time Objeto, T do time Fundo. Todo pixel começa ligado aos dois capitães; ligação grossa quer dizer que o pixel combina com aquele time. No fim, cada pixel só pode ficar com um, e recortar a foto vira decidir, para cada pixel, de qual capitão ele se solta.

Os riscos são certezas



Risco vermelho: este pixel é objeto.
Fica preso ao **S** (source).

Risco azul: este pixel é fundo.
Fica preso ao **T** (sink).

A ligação de um pixel riscado com o seu vértice tem peso infinito: ninguém separa.

Primeira fonte de peso para as ligações com S e T: o usuário. As cores dos riscos são as mesmas de S e T no slide anterior. Quem foi riscado de vermelho ganha uma ligação de peso praticamente infinito com S (objeto); de azul, com T (fundo). Esses pixels já têm lado definido.

E os pixels sem risco?



Cada pixel é comparado com as cores que foram riscadas.

Parece o que foi riscado de vermelho: peso alto com S

Parece o que foi riscado de azul: peso alto com T

Segunda fonte de peso: a cor. Para os pixels sem risco, olhamos com que frequência aquela cor aparece nos riscos vermelhos e nos azuis (um histograma de cores). O mapa mostra esse palpite, pixel a pixel. Note os erros: partes azuis da armadura parecem fundo. Sozinho, o palpite por cor não basta; quem conserta são os pesos entre vizinhos, e é isso que o corte vai equilibrar. A conta exata (menos log da probabilidade) está no slide de resumo das contas.

Problema: temos duas pistas, e às vezes elas discordam



Pista 1: a cor

O palpite do slide anterior. Erra onde o objeto tem cor de fundo: as partes azuis da armadura.

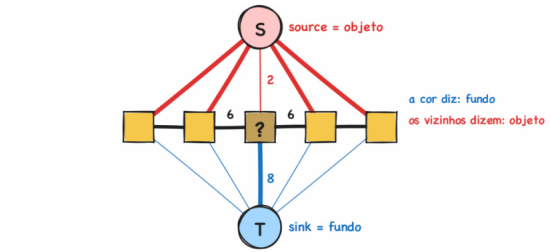
Pista 2: os vizinhos

Pixels colados e parecidos tendem a estar do mesmo lado.

Falta uma regra para decidir cada pixel levando as duas em conta.

Recapitulando o que temos: cada pixel está ligado a S e a T (peso vindo dos riscos ou da cor) e aos seus vizinhos (peso vindo da semelhança). São duas pistas diferentes. A pista da cor olha o pixel sozinho, em qualquer lugar da foto, e erra: as manchas azuis dentro do corpo no mapa são pedaços da armadura que têm a cor do fundo. A pista dos vizinhos é local: diz que esses pedaços estão grudados no resto da armadura. Se ouvirmos só a cor, o recorte sai furado. Precisamos de uma regra que pese as duas.

A regra: separar custa. Vence o jeito mais barato.



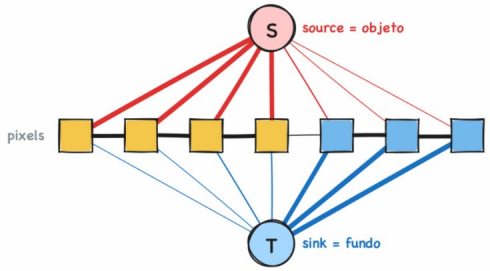
Cada pixel tem de se soltar de **S** ou de **T**.

Vizinhos que ficam em lados diferentes também se soltam.

Romper uma ligação **custa o peso dela**.

Um pixel de exemplo. A cor dele diz fundo: ligação grossa com T (peso 8), fina com S (peso 2). Mas ele está entre vizinhos do objeto, com peso 6 de cada lado. Opção 1, seguir a cor: ele vai para o fundo, e para isso rompe a ligação com S e as duas com os vizinhos: 2 + 6 + 6 = 14. Opção 2, ficar com os vizinhos: rompe só a ligação com T: 8. A regra é escolher o mais barato, então ele fica no objeto. É assim que os vizinhos consertam o erro da cor. E é aqui que entram os dois controles: aumentar o peso entre vizinhos faz a pista 2 ganhar mais vezes.

Fazer isso para a foto inteira tem nome: corte



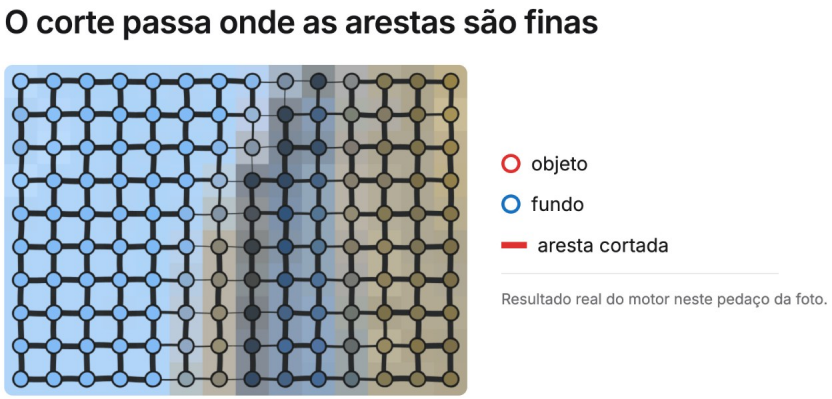
Um corte separa S de T.

É o conjunto de ligações rompidas.
Custo = soma dos pesos.

Corte ruim: **38**. Corte bom: **11**.

O mais barato de todos é o **corte mínimo**. Ele é o nosso recorte.

A mesma regra do slide anterior, agora para todos os pixels de uma vez. Uma escolha de lado para cada pixel é uma linha que separa S de T, e essa linha se chama corte (cut). O custo do corte é a soma dos pesos das ligações que ele rompe. A animação mostra dois: um ruim, que rompe ligações grossas (38), e um bom, que só passa por ligações finas (11). O corte de menor custo possível é o corte mínimo (minimum cut), e é ele que usamos como recorte.



Resultado real do motor neste pedaço da foto. As arestas tracejadas são as cortadas: todas finas.



Cada pixel pode ficar de um lado ou do outro.
Calcular o custo de todos para achar o menor é **impossível**.

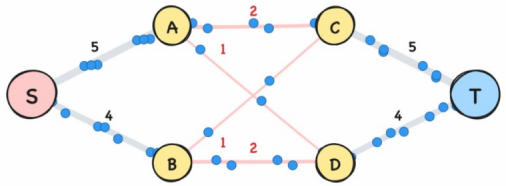
Sabemos o que queremos: o corte mais barato. O problema agora é achá-lo. São 106 400 pixels, cada um com duas opções; o número de cortes tem mais de 32 mil dígitos. Testar um por um não termina nunca. Precisamos de um atalho.

Solução: Lembrar do Prof Todaro



O atalho vem de uma ideia que parece não ter nada a ver com corte. Olhe o mesmo grafo como encanamento: cada aresta é um cano e o peso é a largura (por isso em redes de fluxo o peso se chama capacidade). S é a caixa-d'água, T é o ralo. O fluxo máximo (maximum flow) é o máximo de água por segundo que chega em T. Quem limita são os canos finos do meio: o gargalo. Guarde essa palavra para o próximo slide.

olhar a rede como canos de água



Em vez de testar cortes, abra a torneira.

Cada aresta é um cano; o peso é a largura. **S** é a caixa-d'água, **T** é o ralo.

Fluxo máximo: o máximo de água que chega de S a T.

Quem limita são os canos finos: o **gargalo**.



O atalho vem de uma ideia que parece não ter nada a ver com corte. Olhe o mesmo grafo como encanamento: cada aresta é um cano e o peso é a largura (por isso em redes de fluxo o peso se chama capacidade). S é a caixa-d'água, T é o ralo. O fluxo máximo (maximum flow) é o máximo de água por segundo que chega em T. Quem limita são os canos finos do meio: o gargalo. Guarde essa palavra para o próximo slide.

fluxo máximo
= corte mínimo

Encha a rede de água. Os canos que enchem primeiro, o gargalo, são exatamente o corte mais barato.

Ford e Fulkerson, 1956

O slide mais importante. As duas ideias são a mesma: a quantidade máxima de água que passa é igual ao custo do corte mais barato. Então, em vez de testar cortes, calculamos o fluxo máximo, e para isso existe algoritmo rápido.



MOSCOU, 1970

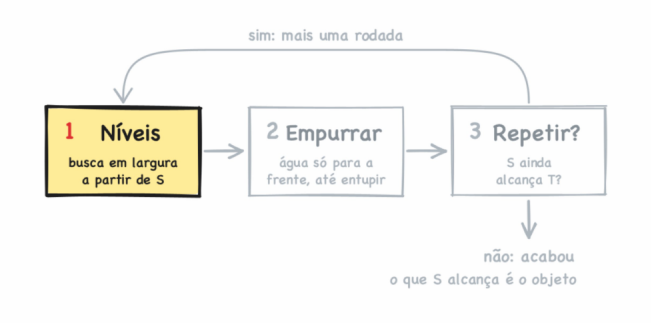
Yefim Dinitz

O problema: achar o fluxo máximo de S a T, rápido.

A ideia dele: organizar a rede em níveis e mandar água por vários caminhos de uma vez, em vez de um caminho por busca.

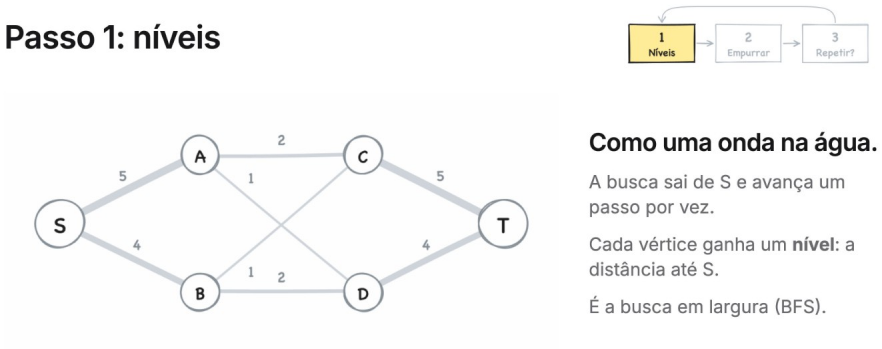
Este senhor é o Yefim Dinitz. Em 1969 ele era estudante de mestrado na Universidade de Moscou, no grupo do Adelson-Velsky (o “AV” da árvore AVL), e criou o algoritmo como resposta a um exercício de aula. Publicou em 1970. O problema era o mesmo do slide da água: dada uma rede de canos, quanto passa de S a T no máximo? Os métodos da época achavam um caminho por vez, com uma busca inteira para cada um. A sacada dele foi fazer uma busca só para separar a rede em níveis e, com ela, empurrar água por todos os caminhos curtos de uma vez. O nome “Dinic”, que aparece nos livros, veio de como o sobrenome foi transcrito quando o algoritmo se espalhou fora da União Soviética. Hoje ele é professor na Universidade Ben-Gurion, em Israel.

O algoritmo de Dinitz é um ciclo de três passos



Nos livros aparece como “algoritmo de Dinic”. Não é o Dijkstra, que acha o menor caminho.

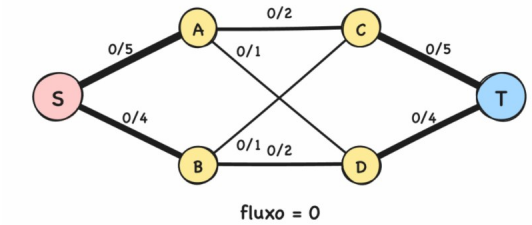
O algoritmo do Dinitz é um laço. Passo 1: uma busca em largura a partir de S dá um nível a cada vértice. Passo 2: empurra água só para a frente, de um nível para o seguinte, até não passar mais nada. Passo 3: pergunta se S ainda alcança T. Se sim, volta ao passo 1 e faz mais uma rodada. Se não, acabou: o que S alcança é o objeto. Os próximos slides mostram cada passo numa rede pequena; o selo no canto mostra em qual passo estamos. Nos livros o nome aparece como Dinic. Cuidado com o nome parecido: Dijkstra resolve menor caminho, outro problema.



Analogia: uma pedra jogada na água em S. A onda chega primeiro em A e B (nível 1), depois em C e D (nível 2), depois em T (nível 3). Isso é a busca em largura, a BFS que já vimos na disciplina. O nível é só a distância até S. Para que serve: no passo 2 a água só poderá andar de um nível para o seguinte, nunca para trás nem para o lado. Isso impede caminhos que dão voltas.

Passo 2: empurrar





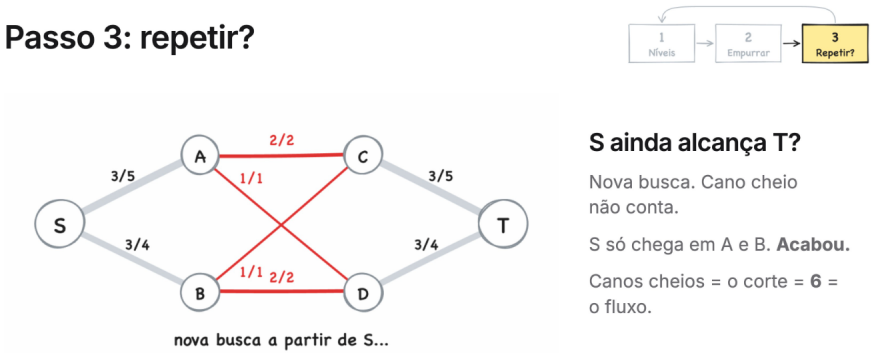
A água só desce a escada.

Um caminho por vez, sempre de um nível para o seguinte.

Cada caminho leva o que cabe no seu cano mais fino.

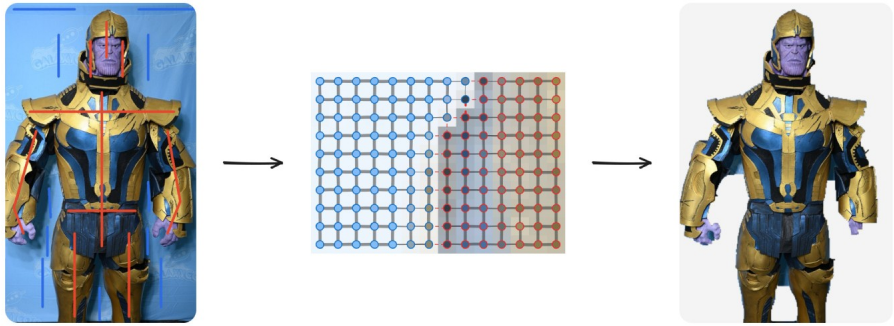
Vermelho: cano cheio.

Analogia: a água só desce a escada, um degrau (nível) por vez. Acompanhe o contador na animação. Quatro caminhos: S-A-C-T leva 2, S-A-D-T leva 1, S-B-C-T leva 1, S-B-D-T leva 2. Cada caminho leva o quanto cabe no seu cano mais fino. Total 6. Os quatro canos do meio encheram e ficaram vermelhos: por eles não passa mais nada.



A pergunta do passo 3: S ainda alcança T? Faz-se uma nova busca a partir de S, mas agora cano cheio não conta. A onda chega em A e B e para: os quatro canos do meio estão cheios. T ficou inalcançável, então o ciclo termina. O que S alcança (S, A, B) é o objeto; o resto é fundo. Os canos cheios que separam os dois grupos são o corte: 2 + 1 + 1 + 2 = 6, exatamente o fluxo que passou. É o teorema acontecendo. Nesta rede bastou uma rodada; numa foto são várias.

Na foto é a mesma coisa



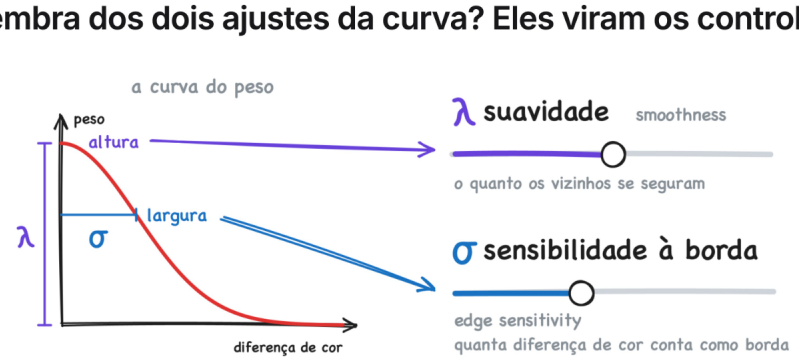
O que S alcança é o objeto. 106 402 vértices, **0,4 segundo**.

Mesmo algoritmo, rede cem mil vezes maior. No fim, uma busca a partir de S: o que ela alcança fica, o resto some.

As contas, em uma página

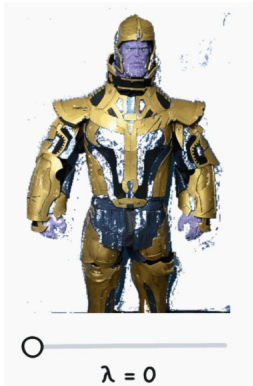
O quê	Conta	Em palavras
Aresta entre vizinhos	$\lambda \cdot e^{\wedge}(-\Delta^2 / 2\sigma^2)$	cores parecidas, aresta forte
Ligação com S ou T	$-\log P(\text{cor})$	parecido com os riscos, ligação forte
Pixel riscado	∞	nunca é cortado
Custo de um corte	soma das arestas cortadas	queremos o menor
Dinic	$O(V^2 \cdot E)$ no pior caso	0,4 s nesta foto

Resumo de todas as contas do projeto. P(cor) vem de um histograma das cores riscadas: 16 faixas por canal. Na prática o Dinic fica muito abaixo do pior caso em grafos de grade.



Voltando ao slide da curva do peso: ela tinha dois ajustes, altura (λ) e largura (σ). Na ferramenta eles viram dois controles deslizantes. λ é a suavidade (smoothness): o peso máximo entre vizinhos, ou seja, o quanto eles se seguram. σ é a sensibilidade à borda (edge sensitivity): quanta diferença de cor é preciso para o peso cair. Agora que já vimos o corte, dá para entender o efeito de cada um; é o que os próximos dois slides mostram. São os dois sliders da janela e as opções --lambda e --sigma do terminal. O padrão é λ = 200 e σ = 40.

λ: suavidade (smoothness)



O quanto os vizinhos se seguram.

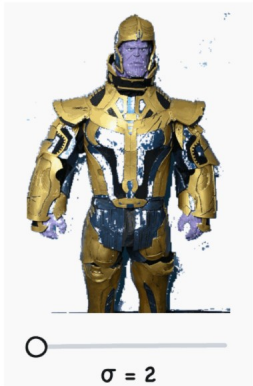
Baixo: cada pixel decide sozinho pela cor. Aparecem buracos e sujeira.

Alto: os vizinhos puxam uns aos outros. O contorno fica liso e inteiro.

Valor padrão: λ = 200

Recortes reais do motor, mudando só λ. Com λ = 0 as arestas entre vizinhos não valem nada e cada pixel segue só o palpite de cor: as partes azuis da armadura somem. Subindo λ, cortar entre vizinhos fica caro e o recorte fecha.

σ: sensibilidade à borda · edge sensitivity



O que conta como borda.

Baixo: qualquer mudança de cor já vira borda. O corte passa por dentro do objeto.

Alto: quase nada é borda. O corte não acha o contorno e leva pedaços do fundo.

Valor padrão: σ = 40

Recortes reais do motor, mudando só σ. σ pequeno: a curva da força cai rápido, então até sombras e dobras da armadura parecem bordas. σ grande: a curva quase não cai, as arestas ficam todas fortes e o corte ignora o contorno verdadeiro.

Lembra do “isso já existe”? Por que eles não fazem assim



Com grafo, alguém precisa **riscar** cada foto e **calibrar λ e σ**.

Rede neural.

Aprendeu, com milhões de fotos, o que é uma pessoa, um cachorro, um objeto.

Sem riscos. Sem λ. Sem σ.

Fechando o que abrimos no começo: remove.bg, iPhone, Photoshop e Canva já fazem isso. Acabamos de ver por que eles não usam o nosso método: o resultado depende de dois riscos e de dois controles que mudam de foto para foto. Ninguém quer calibrar λ e σ para tirar o fundo de uma selfie. Eles usam um modelo treinado, que reconhece o objeto sozinho.

Demo

Fé para não travar



Hora da demo. Roteiro sugerido: (1) ./thanos ui, abrir samples/chupeta, dois riscos, Snap: funciona. (2) Abrir samples/professor, riscar, Snap. (3) Mexer em λ e σ ao vivo para a turma ver o que os dois controles fazem. (4) Mostrar um limite de propósito: num cabelo ou pelo, a borda sai dura, porque cada pixel é objeto ou fundo, sem meio-termo; e onde o objeto tem a cor do fundo, é preciso riscar mais. (5) Terminal: ./thanos cut samples/wazowski/photo.png --scribbles samples/wazowski/scribbles.png --out wazowski.png. Lembrar que o corte roda numa cópia reduzida a 400 pixels, por isso responde em menos de um segundo.

Demo

Fé para não travar



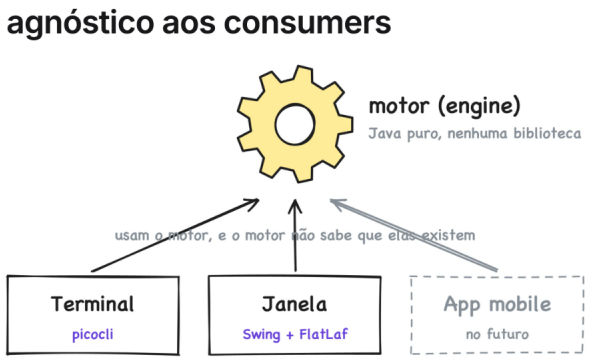
Hora da demo. Roteiro sugerido: (1) ./thanos ui, abrir samples/chupeta, dois riscos, Snap: funciona. (2) Abrir samples/professor, riscar, Snap. (3) Mexer em λ e σ ao vivo para a turma ver o que os dois controles fazem. (4) Mostrar um limite de propósito: num cabelo ou pelo, a borda sai dura, porque cada pixel é objeto ou fundo, sem meio-termo; e onde o objeto tem a cor do fundo, é preciso riscar mais. (5) Terminal: ./thanos cut samples/wazowski/photo.png --scribbles samples/wazowski/scribbles.png --out wazowski.png. Lembrar que o corte roda numa cópia reduzida a 400 pixels, por isso responde em menos de um segundo.

FIM DA TEORIA

Arquitetura do projeto



Virada de assunto: acabou o algoritmo, agora é como o código foi organizado. É rápido.



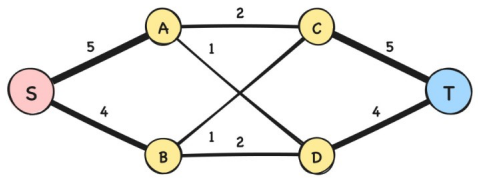
O motor é Java puro. Biblioteca, só nas telas.

O motor (engine) é onde está tudo o que vimos: o grafo, os pesos, o Dinitz. Ele é Java puro, sem nenhuma biblioteca, porque era justamente isso que queríamos construir. Em volta dele ficam as telas, que só o consomem. O terminal usa a biblioteca picocli para ler os comandos. A janela usa Swing, que já vem com o Java, e FlatLaf para o visual. As telas conhecem o motor; o motor não sabe que elas existem. Por isso dá para trocar ou acrescentar uma tela, como um app mobile, sem mexer no algoritmo. Um teste automático verifica essa regra lendo os imports.

O grafo não sabe o que é uma foto

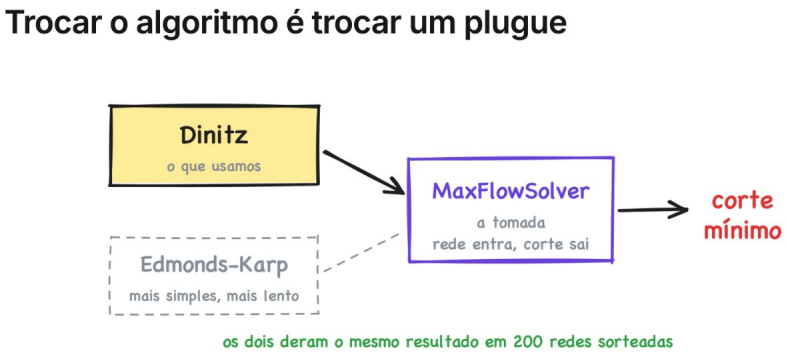


o que a gente vê

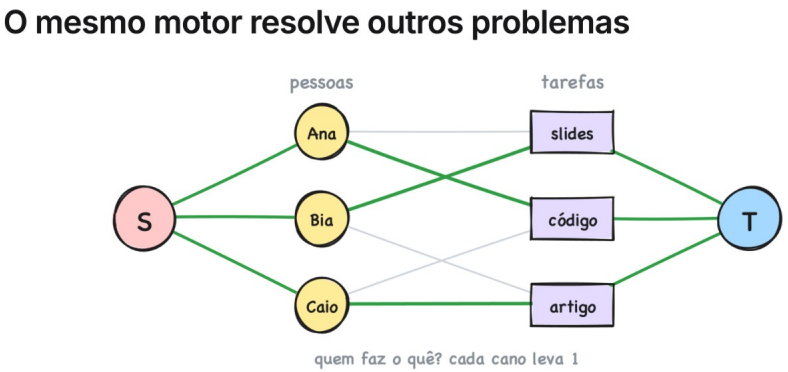


o que o motor do grafo vê: vértices, arestas e números

Dentro do motor há duas peças. Uma conhece fotos: transforma pixels e riscos numa rede e, no fim, transforma o corte de volta em recorte. A outra só conhece redes: recebe vértices, arestas e capacidades e devolve o corte mínimo. Essa segunda peça nunca viu um pixel. Para ela, a foto do Thanos e a rede de seis vértices são o mesmo tipo de problema, só muda o tamanho.



MaxFlowSolver é uma interface Java: recebe uma rede, devolve o corte mínimo. Escrevemos dois algoritmos que cabem nela. O Dinitz é o que o programa usa. O Edmonds-Karp é mais simples e mais lento, e serve de conferência: um teste automático sorteia 200 redes e exige que os dois deem o mesmo resultado. Trocar um pelo outro é mudar uma linha.



Dividir as tarefas de um grupo também é fluxo máximo. E redes de transporte, de dados, de energia.

Para fechar a parte de grafos e suas aplicações. Como o motor só conhece redes, ele serve para qualquer problema que vire uma rede com S e T. Exemplo: três pessoas, três tarefas, cada pessoa só sabe fazer algumas. Liga S a cada pessoa, cada pessoa às tarefas que sabe fazer, cada tarefa a T, tudo com capacidade 1. O fluxo máximo diz quantas tarefas dá para distribuir, e por onde a água passou diz quem faz o quê. O mesmo vale para trânsito, redes de computadores, distribuição de energia e escalas de trabalho.

Perguntas?



Fechar com o Wazowski recortado pelo próprio Thanos.